CONTINUITY

Software Configuration Management Plan

SC3040 Advanced Software Engineering
College of Computing and Data Science
Nanyang Technological University, Singapore

Prepared by Team Continuity

Jerral Chua Fung Yong
Kataruka Mohika
Jethani Samarth
Sinha Armaan
Jain Sparsh
Faye Chan
Lai Yi

Document control	Issue details
Document identifier	CNT-SCMP-001
Revision	01 — Initial issue
Issue date	20 October 2026

TABLE OF CONTENTS

1 Identification 3
1.1 Document overview 3
1.2 Abbreviations and Glossary 3
1.2.1 Abbreviations 3
1.2.2 Glossary 4
1.3 Conventions 4
2 Organization 4
2.1 Activities and responsibilities 5
2.1.1 Decisions process and responsibilities 6
3 Configuration identification 7
3.1 Identification rules 7
3.1.1 Identification rules of configuration items 7
3.1.2 Identification rules of SOUPs 10
3.1.3 Identification rules of documents 11
3.1.4 Identification rules of a media 12
3.2 Reference configuration identification 12
3.3 Configuration Baseline Management 13
4 Configuration control 13
4.1 Change Management 13
4.2 Interface Management 15
4.3 Evolutions control of SOUP items 16
5 Configuration support activities 16
5.1 Configuration Status Accounting 16
5.1.1 Evolutions traceability 16
5.1.2 Setting up Configuration status 17
5.1.3 Configuration status diffusion 17
5.1.4 Configuration status records storage 17
5.2 Configuration audits 18
5.3 Reviews 20
5.4 Configuration management plan maintenance 20

1 Identification

This Software Configuration Management Plan defines how Team Continuity shall identify, version, review, change, release and retain the software and supporting information for Continuity. It is the project's configuration-management reference and shall be linked from the Project Management Plan. It supports the requirements, quality, risk, change-management and release-planning records without replacing their approval responsibilities.

1.1 Document overview

Continuity supports block-level electronic-board design and bill-of-materials validation. An engineer can turn an English brief into a board, select real components, inspect deterministic validation results, repair conflicts, answer an escalation, recover an interrupted run, validate or export a BOM, supply a datasheet, browse retained memory and review validation analytics. Accounts, project ownership, saved boards and guided onboarding support these workflows.

This plan covers the React client, FastAPI backend, LangGraph workflow, pure Python constraint engine, supplier adapters, normalisation and prompts, PostgreSQL persistence, interfaces, tests, deployment configuration and project documents. It also controls the evidence needed to reconstruct a release and explain its results. Runtime user data and credentials require controlled handling; they are not source-code assets to be committed to Git.

Schematic capture, PCB routing, pin assignment, netlist generation, circuit simulation, manufacturing output and horizontal scaling are outside the current product scope. A change that adds one of these capabilities, changes an acceptance criterion or adds another distributor shall follow the scope-change approval process.

1.2 Abbreviations and Glossary

1.2.1 Abbreviations

Abbreviation	Meaning
ABL / FBL / PBL	Allocated / functional / product baseline
API	Application programming interface
BOM	Bill of materials
CI	Configuration item; continuous integration is written in full where ambiguity is possible
CM / SCM / SCMP	Configuration management / software configuration management / this plan
CR / CSA	Change request / configuration status accounting
CSV / PDF	Comma-separated values / Portable Document Format
DS / UC	Design target specification / use case
MCP / MPN	Model Context Protocol / manufacturer part number
MR	Merge request
NRND	Not recommended for new designs
QA / UAT	Quality assurance / user acceptance testing
SOUP	Software of unknown provenance; third-party software subject to dependency control

Abbreviation	Meaning
SQL / SSE	Structured Query Language / server-sent events
SRS / VDD	System Requirement Specification / Version Delivery Description

1.2.2 Glossary

Term	Definition used in this plan
Configuration item	An identified software, data-definition, configuration, test or document asset placed under change control.
Baseline	A reviewed, approved and uniquely identified set of item revisions. A later change creates a new baseline; it does not rewrite the earlier one.
Reference configuration	The exact compatible set of code, contracts, dependencies, prompts, schema state and documents recorded for a review or release.
Frozen run input	The recorded board, normalised physical values and selected supplier observations used for deterministic replay. It is distinct from fetching fresh supplier data.
Prompt fingerprint	The identifier of the normalisation prompt used with an MPN to identify a cached physical-part interpretation.
Release candidate	A fixed configuration submitted for release checks. Candidate status does not establish acceptance or approval.
Version Delivery Description	The release or milestone manifest identifying its contents, approvals, changes, evidence, limitations and recovery instructions.

1.3 Conventions

"Shall" states a required control under this plan; "may" identifies a permitted option. The identifiers, branch names, registers and release formats defined here establish the project's configuration-management conventions. An item becomes baselined only when its exact revision, review evidence and approval are recorded.

Document revisions, software versions, baseline identifiers and runtime run identifiers are separate. UC-1 to UC-16 and DS-1 to DS-16 retain their requirements meanings. Dates in controlled records shall identify the full date; machine timestamps shall include their timezone. Test records shall identify their build and dataset and distinguish execution status (passed, failed, blocked or unexecuted) from the product verdicts they check (pass, fail, warning or unchecked). An unchecked product verdict shall never be represented as a pass.

2 Organization

Configuration management is performed by Team Continuity. Jerral Chua Fung Yong coordinates software configuration and release engineering; Mohika Kataruka manages project decisions and approvals; Sparsh Jain provides technical authority. Verification and validation remain assigned to Samarth Jethani and Faye Chan respectively, with non-author review of changes.

The team shall maintain controlled source and configuration in the campus GitLab repository at https://10.96.189.23/sc3040/TEL1/b, accessible through the NTU network or VPN. GitLab issues shall hold the authoritative change and defect records; Trello shall support work planning and link to those records. Reviewable code and documents shall be version controlled. Large

evidence shall be held in the sponsor-nominated shared drive and linked to the relevant build, test and baseline.

Role and person	Configuration-management responsibility
SCM / release engineering Jerral Chua Fung Yong	Maintain item and dependency registers; repository access; continuous-integration and deployment configuration; release manifests, tags, archives and recovery instructions.
Project manager Mohika Kataruka	Coordinate priorities, change decisions, risks, document control, review minutes and sponsor approval; retain decision, owner and due-date records.
Technical lead Sparsh Jain	Review architecture, engine, rules, repair constraints, interfaces and compatibility; assess technical impact and approve technical readiness.
Backend developer Armaan Sinha	Control backend, accounts, sourcing, adapters, normalisation, datasheet handling, persistence and external-service integration.
Frontend developer Lai Yi	Control client workflows, trace/graph/conflict/BOM presentation, analytics views and interface compatibility.
Verification lead Samarth Jethani	Maintain rule, property, contract, regression and adversarial verification evidence; audit requirements coverage and defect retests.
Validation lead Faye Chan	Maintain browser, system, usability and acceptance evidence; verify the user-visible behaviour of candidate releases.
Sponsor Ajith Santhisenan	Provide written approval for scope or acceptance changes and formal phase acceptance. Routine implementation reviews remain with the team.

2.1 Activities and responsibilities

Each activity shall have a named owner and an independent reviewer appropriate to its impact. The module owner shall nominate a second reviewer and provide a handover when unavailable; the author shall not approve their own engineering change.

Activities when setting up the project	Person responsible
Identify items, owners, dependencies and controlled locations; register source and document revisions.	Jerral with all module owners; Sparsh reviews technical boundaries.
Set up and verify repository access, issue records, review permissions and the reference space.	Jerral; Mohika confirms team and sponsor access needs.
Define review, baseline, change, archive and delivery procedures; link this plan to project records.	Jerral and Mohika; Samarth and Faye review evidence controls.
Record reproducible environment, schema setup, supplier fixtures and dependency versions.	Jerral and Armaan, with Samarth for verification fixtures.

Activities during the project lifecycle	Person responsible
Create a change branch; modify and test the affected items; update relevant documentation.	Assigned owner; independent reviewer.
Review engine and interface changes, their legality constraints and compatibility.	Sparsh; relevant backend/frontend owner.
Place validated revisions under control; assemble a candidate and its VDD.	Jerral; Samarth and Faye verify evidence.
Approve a reference configuration and authorise delivery after required checks.	Mohika after Sparsh's technical review; sponsor for phase acceptance.
Preserve source, configuration and evidence archives and verify their retrieval.	Jerral; evidence custodians verify their records.
Audit the baseline, retest corrective actions and retain findings.	Samarth and Faye; Mohika resolves escalation.

Management activities	Person responsible
Manage versions, baseline register, release records and superseded revisions.	Jerral.
Maintain configuration status, change/defect links and weekly reporting.	Jerral and Mohika; each owner supplies status.
Manage reference-space access, protected credentials and archive copies.	Jerral; Armaan supports service configuration.
Maintain verification and validation reports and acceptance decisions.	Samarth, Faye and Mohika respectively.
Review this plan and coordinate amendments as tools, scope or ownership change.	Jerral drafts; Mohika coordinates approval.

2.1.1 Decisions process and responsibilities

Mohika shall record each change decision after Sparsh assesses technical impact and the affected owner and test lead assess implementation and verification effort. A routine correction within the approved scope may proceed through team approval. A scope, acceptance or milestone commitment change shall be escalated to the sponsor for written agreement before the affected baseline is changed. Rejected and deferred requests shall retain their reasons.

At the end of an activity of the project

Activities	Person Responsible
Freeze a candidate and identify the exact affected component and document revisions.	Jerral with the responsible owners.
Present configuration state, requirements/test links, open defects and known limitations.	Jerral, Samarth and Faye.

Activities	Person Responsible
Record technical readiness, approval conditions and delivery/phase decision.	Sparsh and Mohika; sponsor for formal phase acceptance.

During a configuration management process audit

Activities	Person Responsible
Coordinate the audit, select its baseline and record actions.	Mohika.
Present change records, manifests, access records and document history.	Jerral; Mohika supplies decisions and minutes.
Present independent verification and validation evidence and audit findings.	Samarth and Faye; reviewers shall not audit only their own work.
Assign corrective actions and verify closure before release where required.	Mohika assigns; the independent reviewer verifies.

3 Configuration identification

Configuration identification shall cover every element that can change product behaviour, compatibility, reproducibility or release evidence. Jerral shall link each item to its actual controlled location before the relevant baseline is approved.

3.1 Identification rules

The configuration-item register shall record identifier, name, owner, location, revision, dependencies, affected use cases/design targets, review status and associated baseline. Items with coupled behaviour shall be delivered as one compatible reference configuration even when they have separate item identifiers.

3.1.1 Identification rules of configuration items

3.1.1.1 Identification of a configuration item

The identifier format shall be CNT-CI-<area>-<sequence>, where CNT identifies Continuity, <area> identifies the controlled group and <sequence> is a three-digit number. Identifiers are permanent and shall not be reused. The initial allocation below defines the register to be maintained by this plan.

Item identifier	Controlled content	Owner
CNT-CI-UI-001	React/TypeScript client; account/project screens; guided walkthrough; trace, graph, conflict drawer and BOM; escalation/recovery; saved boards; memory and analytics presentation.	Lai Yi
CNT-CI-API-001	FastAPI routes; authentication/session and ownership checks; projects/runs; design, resume/continue, BOM, datasheet, memory and analytics services.	Armaan

Item identifier	Controlled content	Owner
CNT-CI-WF-001	LangGraph orchestration; sourcing/validation/repair flow; legal-action checks; human interrupts; checkpoint recovery; persistent repair limits and ordered stream production.	Sparsh / Armaan
CNT-CI-ENG-001	Pure Python ten-rule engine; input/units interpretation; verdict and conflict generation; missing-data behaviour and deterministic serialisation.	Sparsh
CNT-CI-SRC-001	JLCPCB and Mouser MCP adapters; manufacturer PDF retrieval; source identifiers, timeouts, errors and provenance mapping.	Armaan
CNT-CI-NRM-001	Normalisation and caching of physical part specifications; numeric extraction; MPN and prompt-fingerprint identity; attachment-based missing-value handling.	Armaan
CNT-CI-PRM-001	Brief interpretation, normalisation and repair-selection prompts; declared model identity; prompt fingerprints and response contracts.	Sparsh / Armaan
CNT-CI-DAT-001	PostgreSQL schema and startup idempotent SQL; data definitions for accounts, owned projects, runs, boards, parts, findings, checkpoints, events and counters.	Armaan
CNT-CI-IFC-001	API request/response and error contracts; board/part/rule schemas; provenance; SSE event types, sequence/replay rules; CSV and analytics output definitions.	Sparsh
CNT-CI-ANL-001	Stored-event analytics calculations and four views; completed-run filtering; denominators and missingness; generated-event analytics seed procedure.	Lai Yi
CNT-CI-TST-001	Test cases, expected results, rule/property/contract/browser/adversarial suites, supplier fixtures, seeded data and requirements-coverage mappings.	Samarth / Faye
CNT-CI-DEP-001	Runtime/build manifests, dependency pins and lock records, third-party inventory, installation and build instructions.	Jerral
CNT-CI-OPS-001	Continuous-integration/deployment settings; environment configuration definitions; secret references; health/monitoring settings; runbook and recovery instructions.	Jerral

Item identifier	Controlled content	Owner
CNT-CI-DOC-001	Requirements/use cases; architecture and maintainability records; quality/risk/configuration/change/release plans; backlogs, decisions, minutes and acceptance evidence indexes.	Mohika

Functional coverage of the controlled configuration

Use case	Capability to preserve	Main item groups
UC-1	Register, log in, restore a session and log out; preserve account access rules.	UI, API, DAT
UC-2	Create an owned project.	UI, API, DAT
UC-3	Browse owned projects and their runs.	UI, API, DAT
UC-4	Rename or delete an owned project.	UI, API, DAT
UC-5	Complete the guided walkthrough and its demonstration-design flow.	UI, API, WF
UC-6	Convert an English brief into slots/rails, real parts and validated board results.	UI, API, WF, ENG, SRC, NRM, PRM
UC-7	Inspect live or saved validation traces, component graphs, conflicts and BOM information without changing the run.	UI, IFC, WF
UC-8	Apply bounded legal automatic repairs and revalidate the whole board.	WF, ENG, PRM, DAT
UC-9	Answer a human escalation and resume from its interrupt.	UI, API, WF, DAT
UC-10	Continue an abandoned or error run from its saved checkpoint.	UI, API, WF, DAT
UC-11	View a saved board without editing it.	UI, API, DAT
UC-12	Validate an uploaded CSV or pasted MPN-based BOM.	UI, API, ENG, SRC, NRM
UC-13	Export the final post-repair BOM as CSV.	UI, API, IFC
UC-14	Attach a PDF with MPN/package information to supply a missing thermal value and its quoted source line.	UI, API, SRC, NRM
UC-15	Browse retained parts, boards and findings in memory.	UI, API, DAT
UC-16	Review the four analytics views for completed owned runs.	UI, API, DAT, ANL

The item-group abbreviations in the table refer to the area segment of the identifiers above. Tests and interface contracts apply across all use cases. A requirement-to-test matrix shall retain positive, negative, empty-state, missing-data and ownership cases where applicable; grouping features into an item shall not remove their individual traceability.

Validation and workflow behaviour to baseline

Rule	Controlled meaning
R1 voltage_overlap	The assigned rail voltage shall fall within the part's supported voltage interval.
R2 current_budget	The rail current budget shall retain the specified 15% margin.
R3 thermal_dissipation	Thermal calculations shall use the required dissipation, thermal-resistance and temperature inputs.
R4 interface_role_match	Parts shall have matching protocol families, complementary roles and compatible signal levels, or an appropriate level translator.
R5 pin_budget	Required pins shall fit the available budget; a shared bus is counted once.
R6 availability	A stock shortfall fails; NRND or obsolete lifecycle status produces a warning.
R7 temperature_rating	The operating requirement shall be checked against the part's temperature rating.
R8 footprint	A footprint concern is advisory and produces a warning.
R9 energy_budget	Battery capacity and continuous draw shall be checked against required runtime.
R10 rail_coverage	A part not connected to any power rail shall be reported unchecked, never pass.

The four outcomes are pass, fail, warning and unchecked. When a rule-required field is absent, the rule shall return unchecked, name the missing field and retain its source context; it shall not substitute a measured default. The model shall not invent an engineering value or decide a verdict. Accepting a conflict shall preserve the acceptance record and continued evaluation of the rule, with the resulting conflict recorded as a warning rather than a pass. The >80% rating and >60°C rise advisories shall not be introduced as additional rules.

The repair vocabulary is swap, change_topology, add_part, change_rail, relax_requirement and escalate. Explicitly requested slots permit only swap, change_rail and escalate. Every selection shall pass the legal-action membership check before application, followed by whole-board revalidation. The limits of three applied repairs per slot and three newly added slots per run, their persisted counters, and a permitted ranked fallback for invalid or timed-out selections shall be controlled together. Up to three earlier repairs recorded for the same conflict signature, applied and subsequently held, may provide context; each proposed action must still pass the legal-action check and whole-board revalidation.

Analytics shall preserve rule-outcome distributions, repair effectiveness, recurring conflict signatures, and supply exposure comprising lifecycle mix and single-source part count. Queries use stored observations from completed owned runs, exclude abandoned/error runs, and retain counts, denominators and unchecked coverage. They shall not invoke a model, fetch new supplier data or revalidate historical boards. The analysis shall remain read-only and shall not persist derived scores.

Persisted user records shall be controlled through schema, access, retention and recovery procedures. Sanitised fixtures and generated test datasets may be versioned for reproducibility.

The analytics seed script identified in the project records, tools/seed_analytics.py, belongs with the controlled seed procedure; generated samples shall remain identifiable as test data.

3.1.1.2 Version number of a configuration item

A source item is identified by its repository location and immutable Git commit. The whole application release shall use <major>.<minor>.<patch>: major for an incompatible contract or data-format change, minor for a compatible capability addition, and patch for a compatible correction. Candidate releases append -rc.<n>. Jerral shall allocate an unused version and record the exact commit before delivery; no version or tag shall be moved to different content.

A release tag shall use v<major>.<minor>.<patch>, with the candidate suffix where applicable. A document-only correction increments its document revision and updates the manifest when delivered. A schema, prompt, fixture, rule or environment change shall identify its own revision and its effect on the application baseline; a software version alone is insufficient to reproduce it.

3.1.2 Identification rules of SOUPs

3.1.2.1 Identification of a SOUP

Third-party packages, runtimes and externally managed services shall be recorded in a dependency register. Each entry shall identify supplier/package name, purpose, source, exact version or available service/model identifier, licence or service terms, owner, configuration references and the baseline in which it is used. Credentials shall be referenced by protected location, never copied into the register.

Dependency family	Technology baseline and control
Frontend	React 19, TypeScript, Vite and Tailwind. Record exact package and build-tool versions from the release manifests/locks.
Backend and workflow	Python 3.11 or later, FastAPI and LangGraph. Record the chosen interpreter and exact installed dependency set.
Model	GLM-5.2 is the model baseline. Record the actual model/service identifier, provider configuration and prompt revisions used for a candidate.
Supplier integration	MCP adapters for JLCPCB, Mouser and manufacturer datasheets. Record adapter/library versions and external contract assumptions.
Persistence and hosting	Managed PostgreSQL and the selected application host. Record the actual database/runtime versions and environment settings before release.
Verification	pytest, pytest-cov and Playwright, with the versions and execution environment used to produce evidence.
SCM and delivery tools	Campus GitLab, issue tracking, Trello and shared-drive evidence storage; record the project locations and access responsibilities.

The pure Python constraint engine shall remain free of third-party runtime dependencies and model, network or system-clock calls. Supplier and model services are external dependencies under this plan; recording them does not make their changing upstream content an immutable software package.

3.1.2.2 Version number of a SOUP

Use the supplier's published version and immutable package or commit identifier where available. Preserve the lock/installation record and integrity information needed to reinstall that dependency. A major version range such as React 19 or Python 3.11+ describes a selection constraint and shall not substitute for an exact release inventory.

For managed services without a fixed release identifier, record the service/model identity, relevant API contract, configuration, observation date and tested behaviour. Supplier stock, lifecycle and datasheet links shall be obtained afresh for each run; the selected observations and their timestamps shall then be retained for replay. Recorded supplier responses used in tests shall be versioned separately from live-service observations.

3.1.3 Identification rules of documents

3.1.3.1 Description of documents identifiers

Controlled document identifiers shall use CNT_<type>_<number>_R<revision>, where <number> is a three-digit sequence within a document type and <revision> is a two-digit issue number. The stable document identity excludes the revision suffix. This plan is CNT_SCMP_001_R01; the running header uses the equivalent stable label CNT-SCMP-001 and Version 01.

Document type	Records covered
PROP / SRS / UC / ICD / SDD	Proposal, requirements, use cases, interface contract and system design.
SCMP / QP / RMP / CMP / RLP	Configuration, quality, risk, change-management and release plans.
MNT / TPD / COV / VEP	Maintainability report, test procedures, test/coverage reports and validation evidence pack.
VDD / RUN / UG / FR	Delivery description, deployment runbook, user guide and final report.
BKL / MIN / DEC	Backlog, meeting minutes and decision records.

Existing document filenames, revisions and deliverable identifiers shall be mapped to the register while preserving their original identity and recorded approval status. A document record shall include title, revision, author, reviewer, status, date, milestone, location and associated change request.

3.1.3.2 Definition and evolution of the revision index

The first issued revision under this scheme is R01. Increment the revision before distributing changed content; retain draft changes in version control until review is complete. The revision history shall describe the change, its origin, author, reviewer and decision date. Approval dates and signatures shall be entered only after the actual review.

Approved revisions shall remain retrievable. A superseded document shall point to its replacement, while the baseline that used it continues to refer to the original revision. Correcting a filename, page reference or formatting issue shall not erase the earlier issued file. The status "for review", "approved", "superseded" or "archived" shall remain separate from the revision number.

3.1.4 Identification rules of a media

A delivery medium is a controlled digital package or archive. Source, deployment, document and evidence bundles shall each have a manifest and an identified storage location. Private runtime data and secret values shall not be included in a general submission or source bundle.

3.1.4.1 Internal identification

The media identifier shall be <baseline-id>/<media-type>/<volume>, where media-type is SRC for source, DEP for deployment material, DOC for documents or EVID for evidence, and volume is a two-digit sequence starting at 01. Each manifest shall record its identifier, filename, baseline, creation date, custodian, contents, integrity checksum and authorised location. Multiple volumes shall be numbered distinctly; copying a package shall preserve its content identity and record the copy location.

3.2 Reference configuration identification

A reference configuration shall use CNT-BL-<type>-<YYYYMMDD>-<sequence>, with type FBL, ABL or PBL and a two-digit sequence for the date. Jerral shall prepare a VDD at each design/test review and for every release candidate. Its identifier and approval record shall identify precisely which configuration was reviewed.

Manifest element	Required record
Identity and authority	Baseline and release/candidate identifiers; creation date; purpose and milestone; preparer, reviewers, approval status and decision links.
Software and build	Repository and immutable commit/tag; item revisions; build/install instructions; frontend/backend runtime and dependency inventory; checksums of delivered packages.
Behaviour and interfaces	SRS/use-case/design-target revisions; rule and repair policies; API/data/event/CSV/analytics contract revisions; model identifier and prompt fingerprints.
Data and environment	PostgreSQL/schema state and startup SQL revision; compatibility with persisted runs/events/checkpoints; non-secret environment configuration; supplier adapters and fixture/seed revisions.
Verification and validation	Build-specific test runs; UC/DS coverage matrix; code-coverage scope and results; determinism, missing-data, repair, security, stream, browser and analytics checks; acceptance evidence.
Change and release assessment	Included change/defect IDs; approved differences from the preceding baseline; unresolved defects and limitations; deployment and recovery procedure; previous compatible configuration.
Delivery and retention	Document revisions; package/media IDs and storage locations; evidence custodian; second-copy and retrieval records; required sponsor phase decision.

A manifest shall not use an unqualified “latest” dependency, mutable branch name or blank approval field as evidence of a fixed baseline. Jerral shall reconcile the manifest with the actual candidate immediately before delivery. A mismatch shall be corrected and reviewed before the candidate is released.

3.3 Configuration Baseline Management

Baseline	Content and establishment	Change control
Functional (FBL)	Scope, UC-1–UC-16, DS-1–DS-16, rule meanings, acceptance criteria and agreed requirement decisions; established at the requirements review.	Technical impact review; written sponsor approval for scope or acceptance changes.
Allocated (ABL)	Architecture, module responsibilities, API/data/event contracts, persistence, prompts/model boundary, supplier interfaces and test strategy; established at design/contract review.	Review all affected producers, consumers and stored-data compatibility; update tests and linked requirements.
Product (PBL)	Exact source/build, dependencies, schema/configuration, tests and evidence, deployment/runbook and user documents; established for an accepted candidate/release.	No replacement in place. An approved change creates a new candidate and product baseline with retained prior records.

A baseline moves from candidate to approved only when review findings, required checks and approval conditions are satisfied. The register shall retain rejected candidates and superseded approved baselines with their reasons and replacement links.

Each iterative release shall reconcile the three baseline layers: the implemented product must trace to the allocated design and functional requirements. A change to one layer shall identify the dependent items and evidence that require revision. Freeze means fixing the reviewed content; subsequent development may continue on a separate change branch without altering that content.

4 Configuration control

Changes shall be traceable from their request to the affected items, review, verification and delivered baseline. Source, rules, prompts, schemas, dependencies, environment settings, interfaces, documents and test fixtures are subject to this process. A release shall contain only the changes listed in its VDD. A discovered deviation from the recorded configuration shall be treated as a change or defect and assessed before further delivery.

4.1 Change Management

Change request record

GitLab issues shall provide the authoritative request identifier. A linked Trello card may represent the work, but the decision, technical evidence and closure record shall remain traceable to the issue. The request shall capture the following information.

Field group	Required information
Identification	Issue ID, title, requester, date, type (defect, enhancement, configuration or document change) and affected baseline/build.
Reason and behaviour	Problem or proposed improvement; expected and actual behaviour; reproducible steps and sanitised evidence for a defect.
Scope and impact	Affected CI IDs, UC/DS requirements, interfaces, prompts/cache, schema/data, dependencies and documents; compatibility and security implications.

Field group	Required information
Delivery assessment	Effort, priority/severity, owner, dependencies, target milestone, schedule/cost/risk effect and any sponsor decision required.
Verification and recovery	Acceptance conditions; affected and regression tests; required dataset/fixture; deployment and reversal approach.
Decision and closure	Approve/defer/reject decision with reason and authority; branch/commits/MR; reviewer; test evidence; retest result; delivered baseline and closure date.

Control workflow

- Register and triage: the owner records the request. Mohika and the test leads prioritise it during weekly triage. Wrong-pass behaviour, a security exposure, data loss or a blocking defect is escalated immediately.
- Assess: Sparsh and affected owners identify behaviour, contract, schema, prompt, test and release impacts. Samarth/Faye define the evidence needed to accept the change. Unresolved requirements are decided before their implementation is approved.
- Decide: Mohika records the team decision and obtains written sponsor approval for a scope or acceptance change. Rejected/deferred requests remain visible. Approved work receives an owner and target milestone.
- Implement: create a short-lived branch from the recorded default integration branch. Use <type>/<issue-number>-<short-description>, where type is feature, fix, docs, chore or hotfix. Link commits and the MR to the issue; update all affected controlled items.
- Review and verify: an engineer other than the author reviews the change. Sparsh additionally reviews engine or contract changes. Required tests and quality gates shall pass before merge, with independent retest of defects.
- Integrate and baseline: merge the reviewed change, record the resulting commit and rerun the affected integration checks. Jerral updates the register and VDD; approved release content receives a new immutable candidate/version.
- Close and communicate: record evidence and the baseline containing the change. A defect closes only after successful retest; failed retest reopens it. Mohika records any remaining action and informs the affected owners.

Defects shall use Open → Triaged → In progress → Ready for retest → Closed, with reopening on failed retest. Severity shall be Critical, High, Medium or Low. Change approval and delivered status shall be recorded separately so that approval is not confused with implementation or release.

Continuous-integration checks shall enforce the agreed coverage gates of at least 80% backend and 95% engine coverage and the required regression/contract checks. Jerral shall verify and record the blocking gate before relying on it for merge or release decisions. While enforcement is being established, the same checks require retained execution evidence and explicit review; missing evidence is not a passing check.

Emergency changes shall retain an issue, impact assessment, independent review and targeted verification. Mohika and Sparsh shall record the urgency, release decision and any additional regression work required. An emergency shall not permit a known false pass or serious security defect to be waived into acceptance. A rollback shall identify the earlier compatible code, configuration and schema state rather than assuming that reverting code alone restores compatibility.

4.2 Interface Management

Sparsh shall coordinate interface change control with both producers and consumers. The contract record shall identify request and response fields, units, optional/missing values, ownership rules, error outcomes, compatibility and tests. A contract change shall update the relevant frontend/backend/engine implementations and tests together.

Interface	Configuration controls and review focus
Client ↔ FastAPI	Account/session and project ownership; design submission; escalation versus abandoned/error continuation; saved boards, BOM validation/export, datasheet attachment, memory and analytics; explicit empty/error states.
Workflow ↔ engine	Normalised board/part input; units and provenance; four verdict outcomes; legal repair membership and bounded counters; whole-board revalidation. Models do not override deterministic verdicts.
Backend ↔ SSE client	Event schema and order; single writer and persisted sequence high-water mark; numbered replay/deduplication; checkpoint compatibility. Heartbeats remain unnumbered and stream failures must be visible.
Adapters ↔ suppliers/PDFs	MPN/package identity, numeric fields and units, stock/lifecycle/datasheet observations, source references and timestamps, timeouts/rate limits and malformed response handling. Unexpected schemas fail explicitly.
Backend/workflow ↔ PostgreSQL	Startup idempotent SQL; schema and state compatibility for accounts, projects, saved boards, events, checkpoints, counters and retained memory. Record a corrective schema procedure.
Stored runs ↔ analytics/export	Completed-owned-run filters; rule outcomes, repair events and conflict signatures; supply exposure; denominators and missingness; final post-repair CSV content and its agreed column contract.

Stable physical specifications are cached by MPN and normalisation-prompt fingerprint. A prompt change that affects interpretation shall force reparse under the new fingerprint. Live stock, lifecycle and datasheet links shall remain fresh for each run and be frozen only as that run's observations. Test fixtures shall not be substituted for live observations without identifying the execution as a fixture-based test.

Open SRS decisions shall remain linked to the affected contract. These include project deletion and list behaviour; registration/session semantics; disconnection, cancellation and saved-board responses; duplicate MPN handling; standalone-BOM memory retention; per-row verdict/unchecked CSV columns; datasheet identity/variant/response details; repair relaxation policy; and the proposed analytics withholding threshold. A change needing one of these decisions shall record the agreed behaviour and tests before baselining it. The proposed 20% unchecked threshold shall not be treated as an approved policy by default.

Thermal ambient temperature, converter efficiency and the build quantity used for stock checks also require recorded decisions. Missing required inputs shall not be replaced with assumed numeric defaults. Mohika shall obtain sponsor confirmation of the delivered scope and acceptance coverage for memory browsing, guided onboarding and standalone BOM validation, and update the proposal accordingly before those scope decisions are treated as approved.

4.3 Evolutions control of SOUP items

A third-party upgrade, service change or model/prompt replacement shall be handled as a controlled change. Armaan owns supplier/model compatibility assessment; Jerral owns package/runtime and service-configuration records. Sparsh reviews effects on deterministic interpretation, interfaces and repair constraints. The selected version shall be tested before it replaces the baseline dependency.

- Record the old and proposed identity/version, source, licence or service terms, reason for change, relevant security/support information and any cost approval.
- Assess API/schema, units, authentication, rate-limit, error, package/runtime and persistence compatibility; identify affected prompts, cached interpretations and reproducibility assumptions.
- Update manifests/locks and the dependency register; retain the prior compatible set. Secrets remain in protected service/environment settings.
- Run relevant unit, contract, integration, supplier-fixture and browser checks, plus live smoke checks where the change depends on a live service. Re-run determinism, missing-data and adversarial tests where interpretation or model behaviour can change.
- Update the VDD, configuration records, recovery/runbook steps and evidence. Approve the candidate through the normal change and release process before adoption.

Model selection and prompt changes shall be assessed together; changing a model shall not bypass the legal-action fence, repair limits or deterministic engine. A changed prompt fingerprint shall prevent reuse of an interpretation produced under an older normalisation prompt. Adding a third distributor or changing an acceptance commitment requires written sponsor approval. Routine compatible dependency corrections remain subject to technical review and regression evidence.

5 Configuration support activities

Configuration support shall maintain a reliable account of what exists, what changed, what was reviewed and what was delivered. Records shall remain linked across the repository, issue tracker, test evidence, documents and release archive so that a reviewer can reconstruct a baseline without relying on an individual developer's memory.

5.1 Configuration Status Accounting

Jerral shall maintain the item and baseline registers, with owners updating records as changes progress. Mohika shall maintain decisions and approval records. Samarth and Faye shall maintain the verification and validation evidence referenced by those registers.

5.1.1 Evolutions traceability

The minimum trace shall be requirement or defect → change request → affected item/document revision → commit/MR → review and test evidence → baseline/VDD → delivery decision. Existing UC and DS identifiers shall be preserved. One change affecting several items shall list every affected item; one item serving several use cases shall retain each relevant requirements link.

Source changes shall carry an issue reference and a meaningful explanation of behaviour. Document changes shall identify affected sections and the revision-history entry. Prompt, schema, fixture and environment changes shall identify their old and new revisions and the evidence invalidated by the change. Release notes shall list the included requests and remaining limitations, and withdrawn or reverted changes shall remain traceable.

5.1.2 Setting up Configuration status

For each item, the register shall contain its label, revision/commit, owner, location, creation/change date, review status, approved baseline and replacement history. For each VDD, record its creation date, author, candidate/release version, included items, evidence links and approval status. Candidate, approved, superseded and archived states shall be explicit.

Status measure	How it shall be reported
Inventory completeness	Items with an identified revision, owner and location divided by the items required for the candidate; list missing records.
Change and defect status	Open/approved/in-progress/ready-for-retest/closed changes; defects by severity and age; reopened and recurring defects; overdue corrective actions.
Requirements coverage	UC-1–UC-16 and DS-1–DS-16 mapped to identified tests/evidence; report passed, failed, blocked and unexecuted obligations with a stated denominator.
Code coverage	Backend and engine percentages from the named build, tool and scope; compare with the 80%/95% gates separately from requirements coverage.
Baseline and release readiness	Unapproved items, failed gates, unresolved audit findings, compatibility/recovery issues and pending review/phase decisions.
Evidence and recovery readiness	Evidence archived against the correct build; second-copy/retrieval status; availability of the matching installation/schema/recovery instructions.

A percentage shall identify its denominator, date, build and execution context. Generated seed runs shall not be counted as user-validation participants. Live-service timing, recorded-response timing and hosted availability shall be reported separately. A dated result from a previous configuration shall not be carried forward as current proof without assessing whether the change invalidates it.

5.1.3 Configuration status diffusion

Jerral and the verification/validation leads shall prepare the configuration and quality status for weekly team review and for each milestone. Mohika shall circulate decisions, actions, owners and due dates and coordinate sponsor communication. Milestone materials shall be circulated at least 48 hours before the review.

The current VDD, register and evidence index shall be available to the team, with sponsor read access to the relevant review material. Release communication shall identify the exact version, included changes, known limitations and deployment/recovery instructions. Wrong-pass behaviour, a serious security issue or a blocking release defect shall be escalated immediately rather than waiting for the next weekly report.

5.1.4 Configuration status records storage

The reference space shall contain change/defect records indexed by identifier, controlled software and documents, VDDs and baseline states in chronological order, review/audit records, and linked verification/validation evidence. Git shall retain code, schemas, tests, infrastructure/configuration definitions and reviewable documents. Large evidence shall be stored in the sponsor-nominated shared drive with build/test/baseline identifiers and an index in the reference space.

Team write access and sponsor read access shall be maintained as appropriate to the records. Secret values shall be stored only in protected environment/service settings and excluded from code, fixtures, logs and exported evidence. Participant results shall use anonymous identifiers; consent/contact information shall be restricted separately. Production user data shall not be included in ordinary source or evidence archives.

Jerral shall archive milestones and release evidence before continuous-integration retention expires, keep a second copy of the release-evidence bundle, and verify retrieval access before handover. Samarth is custodian of verification records, Faye of validation/UAT records, Jerral of release archives, and Mohika of decisions/sign-offs. Evidence shall be retained for one year after final handover, after which deletion or transfer shall be reviewed with the sponsor.

Repository continuity shall include current local clones and recorded instructions for pushing through the NTU network/VPN. Database and run-state recovery shall be addressed separately in the deployment runbook: the actual service backup/export mechanism, access, retention, restoration steps and compatible schema/application revision shall be recorded. Before a schema-affecting release, Armaan and Jerral shall review recoverability and retain the applicable backup/export and non-production recovery-check evidence. Unresolved recovery constraints shall be recorded in the release decision; an evidence-bundle copy shall not be treated as a database backup.

5.2 Configuration audits

Audits shall verify consistency between the approved requirements, the reference configuration, the executable product and its evidence. Samarth and Faye shall perform independent checks within their verification/validation responsibilities; Mohika shall coordinate decisions and corrective actions. Jerral shall present records and resolve configuration discrepancies with the relevant owner.

Audit	Checks and required outcome
Baseline / physical configuration audit	Reconcile item register, commits/tags, package/dependency inventory, prompt fingerprints, schema state, environment definitions, document revisions and delivered checksums with the VDD. Confirm that the package can be installed using its recorded instructions.
Functional configuration audit	Trace UC/DS requirements and agreed contracts to build-specific tests and acceptance evidence. Check rules, missing-data behaviour, repair legality/termination, ownership, streams/recovery, BOM fidelity, datasheet provenance, memory and analytics.
Process and records audit	Sample changes for authority, non-author review, test/defect closure and baseline links; check access/secret handling, retained approvals, archive copies, retrieval evidence and overdue actions.

Release evidence checks

Area	Evidence required for the corresponding acceptance criterion
Rules DS-1	All ten baseline rules covered by unit tests; zero model, network or system-clock access in engine evaluation.

Area	Evidence required for the corresponding acceptance criterion
Determinism, provenance and missing data DS-2, DS-3, DS-4	100 consecutive repetitions of identical frozen inputs with byte-identical verdict sets; separately check brief-to-verdict normalisation drift. At least 40 missing-data boards produce unchecked outcomes naming missing fields, with zero substituted measured defaults. Every verdict retains its deciding field, verbatim value and resolvable source or named reference; missing-field evidence identifies absence and source context.
Repair and adversarial behaviour DS-7, DS-10, DS-14	10 unsatisfiable cases terminate within the repair limits; at least 25 adversarial briefs yield no false pass; zero unvalidated repairs accepted, with legal-action and whole-board revalidation evidence.
Coverage and integration DS-8	At least 80% backend and 95% engine executable-line coverage using pytest-cov, with tested commit, scope and exclusions; UC/DS mapping and relevant regression, contract, browser and recovery results.
Streaming DS-6	First event within two seconds of submission; zero sequence gaps or duplicates across reconnect, interrupt/resume and abandoned/error continuation; explicit failure instead of a silent blank or stalled client.
BOM and export DS-9, DS-11	Every BOM item has a real MPN and sufficient stock or an explicit unmet flag. Export matches the final post-repair board and includes MPN, quantity, price, stock and datasheet URL; verify Excel and Google Sheets interoperability.
Security DS-13	Argon2id password hashing, secure session cookies, ownership-isolation tests and zero committed secrets, supported by configuration inspection and repository scans.
Performance DS-5	For boards of up to 12 slots, evidence against the 130-second live and 35-second recorded-response targets, with execution conditions identified.
Usability and availability DS-12, DS-15	At least 8 of 10 first-time users independently complete the design/conflict-identification task within ten minutes, using the walkthrough and CSV export. Hosted availability is at least 99% over two weeks, cold start within 60 seconds and warm health response within 500 ms; missing scheduled probe samples count as failures.
Analytics DS-16	At least 20 seeded completed runs; all four views reproducible by direct queries against stored findings/events; rates retain numerator, denominator and unchecked count; no persisted derived score. The withholding threshold follows the resolved requirement decision.
Release integrity	All DS criteria evidenced; final regression passes; no unresolved wrong-pass, silent blank-client or critical security defect; matching runbook, documents and approval records.

Auditors shall assess the candidate's recorded test results against the acceptance criteria and the applicable approved requirement decisions. A finding shall record the baseline, discrepancy, severity, owner, due date and closure evidence. A failed release gate remains open until corrected

and independently rechecked; changing the gate itself requires the applicable approval. The audit report shall state the exact configuration examined and any limitations of the examination.

5.3 Reviews

Technical and management reviews shall establish or update the relevant baseline and verify that implementation, documents and evidence remain aligned. Jerral shall prepare the configuration state and freeze the reviewed candidate; the owner shall explain changes and compatibility; Samarth/Faye shall present the appropriate test evidence; Mohika shall retain decisions and outstanding actions.

Review point	Configuration purpose
Each merge request	Review the change scope, affected contracts/items, non-author approval and required tests before integration.
Weekly team / risk / defect review	Review configuration status, open changes, severity/age, dependency and credential issues, failed metrics, archive readiness and corrective actions.
Requirements and design/contract review	Establish FBL/ABL content and resolve relevant requirement/interface decisions before the dependent product configuration is accepted.
Engine, integration and validation reviews	Examine the matching rule, contract, browser/system and user-validation evidence; identify which baseline each result supports.
Release and handover review	Confirm the PBL, VDD, acceptance evidence, document set, package/recovery instructions, retained copies and required sponsor decision.

The review record shall identify attendees, baseline/revisions reviewed, evidence, decisions, conditions, action owners and dates. A phase remains open until its exit criteria are demonstrated and the required written sponsor sign-off is recorded. An unresolved action shall either block the affected baseline or be explicitly recorded with an authorised disposition that does not waive a mandatory release gate.

5.4 Configuration management plan maintenance

Jerral shall maintain this plan with Mohika. Sparsh shall review technical controls, Samarth verification/audit provisions, and Faye validation and acceptance-evidence provisions. The team shall review the plan at milestone preparation and whenever ownership, repository/tooling, dependencies, architecture, deployment arrangements, scope or acceptance requirements change.

Amendments shall follow document change control: record the reason and impact, revise affected sections, obtain required review/approval, increment the revision, update the register and notify affected owners. Scope or acceptance changes require sponsor approval. Retain superseded revisions and approval records with their baselines.

Before release, control owners shall verify records of item locations/revisions, merge/gate enforcement, dependency/schema inventories, evidence access and recovery instructions. Track incomplete controls, owners, due dates and release-readiness impact. At handover, transfer the final plan, item/baseline registers, change history, VDDs, archive-access instructions and agreed evidence-custodian responsibilities.